InfraQ

A Scheduling Gateway for Shared Local LLM Inference

InfraQ brings four capabilities to shared local LLM serving on Mac hardware, without CUDA: an async RabbitMQ queue that decouples acceptance from inference; a K-slot parallel scheduler that exploits server-side concurrency; an exact-match Redis cache returning repeated prompts in ∼11 ms; and a live Redis configuration channel for zero-restart strategy switching. On repeated-prompt workloads, cache-aware dispatching achieved ***5.6× higher throughput*** *and* ***76% lower average latency*** *over non-cached strategies; on unique-prompt workloads (OLLAMA_NUM_PARALLEL=4, 3 repeats), parallel strategies achieved* ***60% higher throughput*** *over sequential (0.368 vs. 0.230 req/s) and* ***38% lower P95 tail latency*** *(252 s vs. 409 s), with continuous dispatching consistently edging static batching by eliminating batch-boundary stalls.*

Problem and Idea

Small engineering teams increasingly run shared local LLMs on a Mac server rather than routing queries to cloud APIs. The problems are predictable: without a queue, concurrent requests pile up and latency becomes unpredictable; even with a queue, a serial consumer leaves server-side concurrency idle; repeated prompts get re-inferred from scratch every time; and switching scheduling strategy requires restarting the serving stack.

vLLM [1] and SGLang [2] solve these problems at the GPU kernel level: continuous batching avoids batch-boundary idle time, and prefix caching reuses KV tensors across shared prompt prefixes. Both require NVIDIA hardware and CUDA toolchains that most small teams cannot justify.

InfraQ re-implements both techniques at the infrastructure layer, so they work on any machine running Ollama [3], including Mac M1. The result is four concrete capabilities: an async RabbitMQ queue that decouples acceptance from inference; a K-slot parallel scheduler that exploits server-side concurrency; an exact-match Redis cache that returns repeated prompts in ∼11 ms; and a live Redis configuration channel that hot-swaps strategy without container restarts.

Scheduling Strategies

InfraQ implements four strategies: a sequential baseline, static batching, continuous dispatching, and cache-aware dispatching. Each builds on the previous one and introduces exactly one new mechanism, isolating its effect in the experiments of §4. Table 1 summarises them.

Table 1: The four scheduling strategies in InfraQ.

Strategy	Mechanism	Inspiration	Trade-off addressed
Sequential	One request at a time; no parallelism	Baseline	None — serial baseline
Static batching	Collect N requests; dispatch all; wait for the entire batch to finish	Classic GPU batching	Gains concurrency, but the slowest request stalls completed slots
Continuous dispatching	Maintain K slots; refill immediately when any slot completes	vLLM continuous batching	Eliminates batch-boundary stalls; keeps slots filled
Cache-aware dispatching	Redis lookup before each slot acquisition; SHA-256 key on model + prompt; hits return in ∼11 ms, skipping Ollama	SGLang prefix caching	Near-zero latency for repeated prompts; frees slots for cache misses

Architecture

The gateway (Spring Boot) returns HTTP 202 immediately, writes a record to PostgreSQL, caches status in Redis, and publishes the job to RabbitMQ. None of these operations block on inference. The worker (Python asyncio) consumes from RabbitMQ and applies the active scheduling strategy. Ollama runs natively on the

host and is reached from the worker container via `host.docker.internal`, so Mac Metal GPU acceleration works without container passthrough.

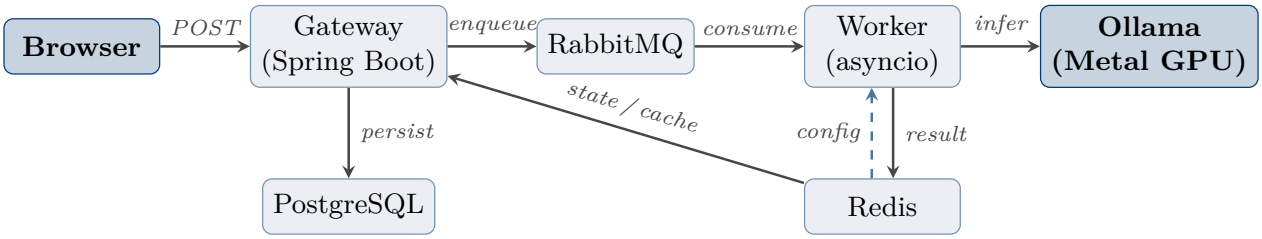

Figure 1: InfraQ system architecture. The top row shows the synchronous request path; the bottom row shows persistent storage and the inference engine. The dashed arrow is the Redis configuration channel through which live strategy changes propagate from the gateway to the worker.

Redis serves four roles: browser-polling fast-path (<1 ms), prompt-result cache, atomic metric counters, and the live configuration channel shown dashed in Figure 1. PostgreSQL stores permanent records (full request histories and per-request timing) for offline analysis. On a configuration change, the worker drains all in-flight tasks then restarts under the new strategy without container restarts.

Experimental Results

MacBook Air M1 (8 GB), Qwen2.5:1.5b via Ollama; 100 requests per run. Experiment A: 3 repeats, `OLLAMA_NUM_PARALLEL=4` (enables true concurrent inference). Experiment B and ablation: 5 repeats, default Ollama settings. With 71% of requests served from Redis, model-server concurrency does not affect the result. Unique runs used 100 distinct prompts (40 topics × 4 templates, no repeats). Repeated runs cycled an 8-prompt hot set for 80% of requests and inserted unique prompts for the remaining 20%, yielding an average 71% cache hit rate. Raw data are in `summary.csv` and `manifest.json` in the submission.

Scheduling without caching (unique prompts)

Table 2: Experiment A — unique prompts (no cache benefit). 3-run averages, OLLAMA_NUM_PARALLEL=4.

Strategy	Slots	Avg latency	P95	Throughput
Sequential	1	174 s	409 s	0.230 req/s
Static batching	8	110 s	263 s	0.364 req/s
Continuous	8	104 s	**252 s**	**0.368 req/s**
Cached	8	108 s	254 s	0.370 req/s

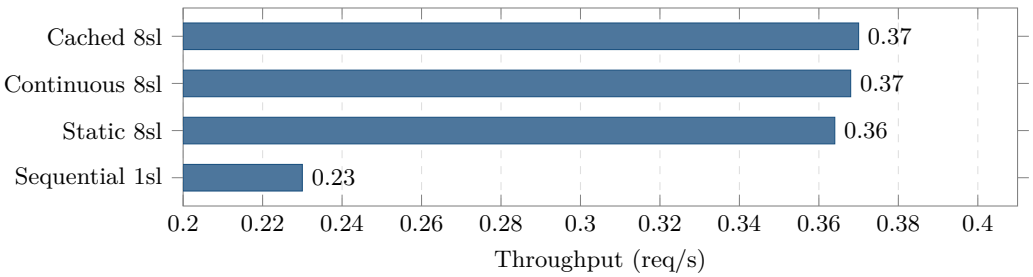

Figure 2: Throughput on the unique-prompt workload (Experiment A, OLLAMA_NUM_PARALLEL=4). Parallel strategies are 60% faster than sequential; continuous dispatching edges static batching by 1% (0.368 vs. 0.364 req/s) by eliminating batch-boundary stalls.

With every prompt unique the cache cannot fire, so the numbers isolate the scheduling mechanism. With `OLLAMA_NUM_PARALLEL=4` enabling true concurrent inference, parallel strategies (static, continuous, cached) reach 0.364–0.370 req/s, versus 0.230 req/s for sequential — a 60% throughput advantage. Continuous dispatching edges static batching on throughput by 1% and on P95 tail latency by 4% (252 s vs. 263 s), consistently across all three repeats. Static batching waits for the slowest request in a batch before refilling any freed slot; continuous dispatching refills the moment a slot frees.

Cache-aware dispatching on repeated prompts

Table 3: Experiment B — repeated prompts (hot prompt set). 5-run averages.

Strategy	Slots	Avg latency	P95	Throughput	Hit rate
Sequential	1	311 s	584 s	0.165 req/s	0%
Static batching	4	298 s	557 s	0.173 req/s	0%
Continuous	4	299 s	565 s	0.170 req/s	0%
Cached	**4**	**73 s**	**96 s**	**0.961 req/s**	**71%**

With 71% of requests hitting the cache, cache-aware dispatching delivers a 5.6× throughput increase (0.961 vs. 0.170 req/s) and reduces average latency by 76% (73 s vs. 299 s) over non-cached continuous dispatching. Each cache hit returns in ∼11 ms rather than the ∼300 s average inference time, a per-hit saving of 96.3%. With only 29 requests per 100 reaching Ollama, slots stay freer and P95 tail latency collapses from ∼565 s to 96 s (83% reduction).

Slot-count ablation (cached strategy)

Increasing the slot count from 4 to 8 under cache-aware dispatching actually *reduces* performance: throughput falls from 0.961 to 0.699 req/s and the hit rate drops from 71% to 67%. The cache starts cold, so the first wave of requests are all misses. With 8 slots, 8 simultaneous misses hit Ollama before any result is written back to Redis; with 4 slots, fewer misses compete for Ollama's capacity and hits accumulate sooner. For cache-aware scheduling, the optimal slot count depends on workload repetition rate and cache warm-up. More concurrency is not always better.

Implementation

All four strategies are implemented, tested, and live-switchable without container restarts. Deployment is a single `docker compose up`; the only external dependency is Ollama with the target model loaded. The web interface exposes a chat tab, a live metrics dashboard, and the benchmark lab that produced the results in §4.

The caching layer is prototype-grade: InfraQ stores exact-match text results rather than KV tensors, so semantically similar prompts still miss. A production system would replace this with embedding-based semantic matching.

AI Usage

All scheduling ideas (continuous batching adapted from vLLM, prefix caching adapted from SGLang) were conceived by the author. System architecture and experimental methodology were designed by the author. ChatGPT and Claude Code assisted with writing code; the web UI was generated by AI from a detailed specification written by the author. This report was polished with AI assistance, and the architecture diagram was drawn with AI tooling.

References

[1] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP '23)*, 2023, pp. 611–626.

[2] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.

[3] Ollama, "Get up and running with large language models locally," `https://ollama.com`, accessed April 2026.